NATIONAL UNIVERSITY

NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

Laboratory Activity 2
Advanced Database Design Using MongoDB

Name	Marzan, Joshua M.
Section	INF 233
Date	15 October 2026

Objectives

- Analyze the data requirements of the Bulldogs Exchange system to identify the necessary database collections and their attributes.
- Design MongoDB collections by applying appropriate data modeling strategies using embedded and referenced documents.
- Develop Mongoose schemas with appropriate data types, validation rules, and collection relationships.
- Implement database indexes on frequently queried fields to improve query performance.
- Validate the designed database models by inserting sample documents and verifying schema relationships.

Materials Needed:

- Visual Studio Code
- Node.js (LTS Version)
- MongoDB Community Server or MongoDB Atlas
- MongoDB Compass (Optional)
- Postman

Prerequisite(s):

- Completed Week 2 Laboratory Activity (Refactored MERN Backend using MVC Architecture)
- Running MongoDB database (Local or Atlas)
- Existing Express.js backend project configured with Mongoose
- Basic understanding of MongoDB collections, JSON documents, and Mongoose schemas

Submission Requirements:

- Filename: Surname_Activity2_CTADWEBL
- Save as PDF

Activity Instructions

Part 1: Analyzing the Bulldogs Exchange Business Requirements
Open the backend project created during the previous Web Programming course. **Verify that (***Provide the Screenshot of your Backend Working*`**):**` • The application runs successfully. • The MongoDB connection is established. • The MVC project structure is intact.

The **marzan-server** backend from the previous Web Programming course (Express.js + Mongoose, organized in the MVC structure from Activity 1) was opened in Visual Studio Code and started with `npm run dev`. Before connecting, the laptop's current IP address was added to the MongoDB Atlas IP Access List (Network Access), so the Atlas cluster accepts the connection.

```
> npm run dev
MongoDB connected
Server running on http://localhost:5000
```

- **The application runs successfully:** `GET http://localhost:5000/` returns `{ "message": "BulldogEx Shop API is running" }`.
- **The MongoDB connection is established:** the console prints `MongoDB connected` (database `bulldogex-shop` on MongoDB Atlas).
- **The MVC project structure is intact:** `components/config`, `controllers`, `middleware`, `models`, and `routes`.

Bulldogs Exchange business requirements

Bulldogs Exchange (BulldogEx Shop) is the online campus store of National University. From its features, the database must support the following:

- **Users** create an account and sign in. Each user needs a name, a unique email, and a securely stored (hashed) password.
- **Products** are sold by the store (not by individual students), so a product has no seller field. Each product has a URL name, title, price, stock status, description, and image.
- **Categories** (Apparel, Stationery, Uniforms, and so on) group the products so students can browse by category. Many products share one category.
- **Cart:** every signed-in student keeps one cart that lists the products and quantities they plan to buy.
- **Orders:** checking out creates an order with the items, shipping address, payment method, payment and delivery status, and total. The price paid must stay the same even if the product price changes later.
- **Reviews:** students rate a product from 1 to 5 and leave a comment, with only one review per student per product.

Screenshot below: VS Code with the components folder expanded and the terminal output of `npm run dev`.

Part 2: Analyze the database requirements of the Bulldogs Exchange system.

Identify the information that should be stored for the following collections:

- Product
- Category
- Cart
- Orders
- Reviews
- User

Complete the table below.

Collection	Required Fields
Product	_id, name (URL slug), title, category (→ Category), price, stock, content (description), image, createdAt, updatedAt
Category	_id, name, description, createdAt, updatedAt
Cart	_id, user (→ User), items [product (→ Product), quantity], createdAt, updatedAt
Orders	_id, user (→ User), items [product (→ Product), title, price, quantity], shippingAddress { fullName, phone, street, city, postalCode }, paymentMethod, isPaid, status, totalAmount, createdAt, updatedAt
Reviews	_id, product (→ Product), user (→ User), rating, comment, createdAt, updatedAt
User	_id, firstName, lastName, email, password (bcrypt hash), createdAt, updatedAt

→ marks a reference to another collection (stored as an ObjectId), [ ] is an embedded array, and { } is an embedded object. `_id` is created by MongoDB, and `createdAt` / `updatedAt` are added automatically by Mongoose ({ `timestamps: true` }).

Fields that must be provided when a document is created: Product – name, title, category, price; Category – name; Cart – user (and product for every item); Orders – user, at least one item, shippingAddress, totalAmount; Reviews – product, user, rating; User – firstName, lastName, email, password. The remaining fields are optional or have default values (for example stock = "In stock", status = "Pending", isPaid = false).

Part 3: Designing the Database Structure

For each collection identified in Step 2, determine the appropriate MongoDB data type for every field.
Examples include:

- String
- Number
- Boolean
- Date
- Array
- ObjectId

Record your answers in a table.

Example:

Collection	Required Fields	
	Field	Data Type
Product	_id	ObjectID
	productName	String

Answer:

Collection	Required Fields	
	Field	Data Type
Product	_id	ObjectId
	name	String
	title	String
	category	ObjectId (ref: Category)
	price	Number
	stock	String (enum)
	content	Array (of String)
	image	String
	createdAt	Date
	updatedAt	Date
Category	_id	ObjectId
	name	String
	description	String
	createdAt	Date
	updatedAt	Date
Cart	_id	ObjectId
	user	ObjectId (ref: User)
	items	Array (embedded documents)
	items.product	ObjectId (ref: Product)
	items.quantity	Number

	createdAt	Date
	updatedAt	Date
Orders	_id	ObjectId
	user	ObjectId (ref: User)
	items	Array (embedded documents)
	items.product	ObjectId (ref: Product)
	items.title	String
	items.price	Number
	items.quantity	Number
	shippingAddress	Object (embedded document)
	shippingAddress.fullName	String
	shippingAddress.phone	String
	shippingAddress.street	String
	shippingAddress.city	String
	shippingAddress.postalCode	String
	paymentMethod	String (enum)
	isPaid	Boolean
	status	String (enum)
	totalAmount	Number
	createdAt	Date
	updatedAt	Date
Reviews	_id	ObjectId
	product	ObjectId (ref: Product)
	user	ObjectId (ref: User)
	rating	Number
	comment	String
	createdAt	Date
	updatedAt	Date
User	_id	ObjectId
	firstName	String
	lastName	String
	email	String
	password	String
	createdAt	Date
	updatedAt	Date

ObjectId is used for `_id` and for every reference field, so a reference has the same type as the `_id` it points to. Number is used for amounts and counts that are computed or compared, Date for the automatic timestamps, Boolean for the paid flag, Array for lists, and Object for the embedded shipping address. Values that must come from a fixed list (stock, paymentMethod, status) are Strings with an enum validator.

Part 4: Analyze the relationships among the collections.

Identify each collection and determine whether each relationship should use:

- Embedded Documents
- Referenced Documents

Provide brief justification for each decision.

Relationship	Strategy	Justification
Product -> Category	Referenced Documents	Products reference categories to avoid redundant data, maintain consistency, and support a one-to-many relationship where multiple products can belong to the same category.
Cart -> User	Referenced Documents	One-to-one: each user has exactly one cart. The cart stores only the user's ObjectId (with a unique index), so the user's profile stays in one place and a user's cart can be found quickly.
Cart -> Cart Items	Embedded Documents	Cart items belong to one cart only and are always read and updated together with it, so they are embedded as an array. Each item still references the Product by ObjectId, so the cart always shows the product's current price and stock.
Orders -> User	Referenced Documents	One-to-many: a user places many orders over time. Embedding them inside the user document would make it grow without limit, so each order stores the user's ObjectId instead.
Orders -> Order Items	Embedded Documents	The items are part of the order and are always displayed with it. The product title and price are copied at checkout (a snapshot), so the order total stays correct even if the product's price changes later. Each item keeps the product's ObjectId to link back to it.
Orders -> Shipping Address	Embedded Documents	One-to-one: the address belongs to that order only and is always shown with it. Embedding avoids an extra query and keeps the address used at checkout even if the user moves later.
Reviews -> Product	Referenced Documents	One-to-many: a product can have many reviews. Keeping reviews in their own collection stops the product document from growing without limit, and an index on product makes listing a product's reviews fast.

Reviews -> User	Referenced Documents	One-to-many: a user can write many reviews. Referencing keeps the reviewer's name in one place (loaded with populate()), and a unique index on { user, product } allows only one review per user per product.

Rule applied: data that is always used together with its parent and has a small, bounded size is embedded (cart items, order items, shipping address). Data that is shared, grows without limit, or is also used on its own is kept in its own collection and referenced by ObjectId (categories, users, products, reviews).

Part 5: Create a Collection Relationship Diagram (CRD)

The diagram should clearly show:

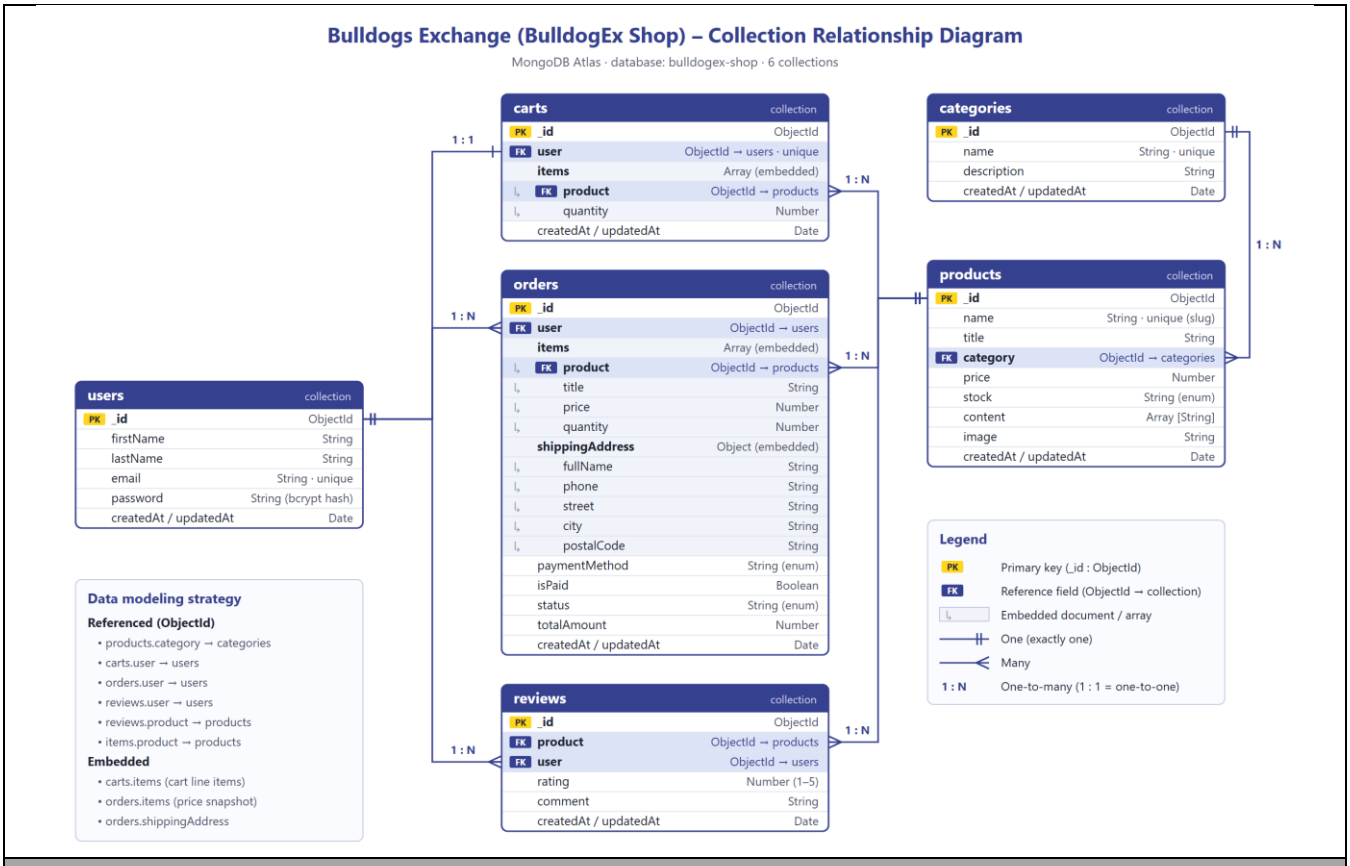
- Collection names
- Relationship types
- Reference fields (ObjectId)

You may use https://draw.io/ or you can use the built-in tool of MongoDB Compass (Data Modeling)

The Collection Relationship Diagram below was drawn from the final Mongoose schemas. Each box is a collection with its fields and data types: **PK** marks the `_id` primary key and **FK** marks the reference fields stored as ObjectId. Each line connects a reference field to the collection it points to, using crow's-foot notation with a **1 : 1** (one-to-one) or **1 : N** (one-to-many) label. Shaded rows (↳) are embedded documents stored inside their parent collection.

- **1 : N** categories → products, users → orders, users → reviews, products → reviews, products → cart items, products → order items
- **1 : 1** users → carts (enforced by the unique index on carts.user)
- **Embedded:** carts.items, orders.items, orders.shippingAddress

Editable source file: `BulldogEx_CRD.drawio` *(open it in draw.io).*



Part 6: Creating Schemas File

Inside the Models folder, create the following schema files:

- productModel.js
- categoryModel.js
- cartModel.js
- orderModel.js
- reviewModel.js
- userModel.js

The six schema files were created inside `components/models`. `Product.js` and `User.js` from the previous activity were renamed to `productModel.js` and `userModel.js` to follow the required file names, and every file that imported them (`productController.js`, `userController.js`, and `seed.js`) was updated. The other four files are new.

```
marzan-server/components/models/
|-- cartModel.js       (new)
|-- categoryModel.js   (new)
|-- orderModel.js      (new)
|-- productModel.js    (renamed from Product.js)
|-- reviewModel.js     (new)
`-- userModel.js       (renamed from User.js)
```

Screenshot below: VS Code Explorer showing the six files inside components/models.

Part 7: Creating MongoDB Schema

Do the following:

- Create a Models/Schema per Collections based on Part 6
- Make sure to implement the relationships between collections
- Make sure to provide screenshots per files created
- You may add more pages if necessary

Each schema is a `mongoose.Schema` with data types, validation rules with custom error messages, and relationships. A relationship is defined with `type: mongoose.Schema.Types.ObjectId` and `ref: "<Model>"`, so related documents can be loaded with `populate()`. Every schema uses `{ timestamps: true }`, which adds `createdAt` and `updatedAt`.

components/models/productModel.js

- **name:** String, required, unique, trimmed, lowercase, and must match a URL slug pattern (for example `campus-tote-bag`).
- **title:** String, required, at most 100 characters.
- **category:** ObjectId with `ref: "Category"`, required (referenced relationship to categories).
- **price:** Number, required, cannot be negative (`min: 0`).
- **stock:** String enum: In stock, Low stock, Out of stock, Preorder (default In stock).
- **content:** Array of String (description paragraphs); **image:** String (image file name).

[Paste screenshot here: productModel.js]

components/models/categoryModel.js

- **name:** String, required, unique, trimmed, at most 50 characters.
- **description:** String, optional, at most 255 characters.

[Paste screenshot here: categoryModel.js]

components/models/cartModel.js

- **user:** ObjectId with `ref: "User"`, required and unique (one cart per user).

- **items:** array of embedded cart items (a sub-schema without its own _id): **product** (ObjectId, `ref: "Product"`, required) and **quantity** (Number, default 1, at least 1, whole numbers only).

[Paste screenshot here: cartModel.js]

components/models/orderModel.js

- **user:** ObjectId with `ref: "User"`, required.
- **items:** array of embedded order items (product with `ref: "Product"`, title, price, quantity), with a custom validator that requires at least one item.
- **shippingAddress:** embedded object, required: fullName, phone (11 digits starting with 09), street, city, and postalCode (4 digits).
- **paymentMethod:** String enum: Cash on Delivery, GCash, Card (default Cash on Delivery); **isPaid:** Boolean (default false).
- **status:** String enum: Pending, Processing, Shipped, Delivered, Cancelled (default Pending).
- **totalAmount:** Number, required, at least 0 (computed by the server, not sent by the client).

[Paste screenshot here: orderModel.js]

components/models/reviewModel.js

- **product:** ObjectId with `ref: "Product"`, required; **user:** ObjectId with `ref: "User"`, required.
- **rating:** Number, required, whole number from 1 to 5.
- **comment:** String, optional, trimmed, at most 500 characters.

[Paste screenshot here: reviewModel.js]

components/models/userModel.js

- **firstName** and **lastName:** String, required, trimmed.
- **email:** String, required, unique, lowercase, and must be a valid email format.
- **password:** String, required (saved as a bcrypt hash by the sign-up controller).

[Paste screenshot here: userModel.js]

Part 8: Database Optimization

Create indexes for fields that are expected to be searched frequently.

Indexes were added to the fields that the API searches, filters, or sorts on most often. Without an index, MongoDB must read every document in the collection (COLLSCAN); with an index it goes straight to the matching documents (IXSCAN). Fields with `unique: true` and every `schema.index()` are created automatically by Mongoose when the server starts. The `_id` field is always indexed by MongoDB.

Collection	Index	Type	Query it speeds up / Justification
products	`{ name: 1 }`	Unique	GET, PUT and DELETE `/api/products/:name` find a product by its URL name on every product page. Unique also prevents two products from having the same URL.
products	`{ category: 1, price: 1 }`	Compound	GET `/api/products?category=<id>` lists the products of one category (category page). Price is included so the products of a category can also be sorted or filtered by price with the same index.

products	`{ title: "text", content: "text" }`	Text	GET `/api/products?search=<keyword>` searches product titles and descriptions by keyword (shop search bar).
categories	`{ name: 1 }`	Unique	Categories are looked up and listed by name. Unique prevents duplicate categories.
users	`{ email: 1 }`	Unique	Sign in and sign up find a user by email on every request. Unique prevents two accounts with the same email.
carts	`{ user: 1 }`	Unique	GET `/api/carts?user=<id>` loads a user's cart. Unique enforces one cart per user.
orders	`{ user: 1, createdAt: -1 }`	Compound	GET `/api/orders?user=<id>` shows a user's order history, newest first, without an extra sort step.
orders	`{ status: 1 }`	Single field	GET `/api/orders?status=Pending` lets the store find the orders that still need to be processed.
reviews	`{ product: 1, createdAt: -1 }`	Compound	GET `/api/reviews?product=<id>` shows a product's reviews, newest first.
reviews	`{ user: 1, product: 1 }`	Unique compound	Checks whether a user already reviewed a product and blocks duplicate reviews (one review per user per product).

Index definitions in the schemas

```
// productModel.js
name: { ..., unique: true },
productSchema.index({ category: 1, price: 1 });
productSchema.index({ title: "text", content: "text" });

// categoryModel.js, userModel.js, cartModel.js
name: { ..., unique: true },   email: { ..., unique: true },   user: { ..., unique: true },

// orderModel.js
orderSchema.index({ user: 1, createdAt: -1 });
orderSchema.index({ status: 1 });

// reviewModel.js
reviewSchema.index({ product: 1, createdAt: -1 });
reviewSchema.index({ user: 1, product: 1 }, { unique: true });
```

Verification

The indexes appear in MongoDB Atlas (Data Explorer → collection → Indexes). Running the main queries with `explain()` shows that each one uses its index (IXSCAN) instead of scanning the whole collection (COLLSCAN):

Query	Winning plan from explain()
`products.find({ category })`	IXSCAN category_1_price_1
`products.find({ $text: { $search: "jersey" } })`	TEXT_MATCH, IXSCAN title_text_content_text
`products.find({ name })`	EXPRESS_IXSCAN name_1

`users.find({ email })`	EXPRESS_IXSCAN email_1
`carts.find({ user })`	EXPRESS_IXSCAN user_1
`orders.find({ user }).sort({ createdAt: -1 })`	IXSCAN user_1_createdAt_-1
`orders.find({ status: "Pending" })`	IXSCAN status_1
`reviews.find({ product }).sort({ createdAt: -1 })`	IXSCAN product_1_createdAt_-1

[Paste screenshot here: index definitions in productModel.js, orderModel.js, and reviewModel.js]

[Paste screenshot here: MongoDB Atlas → Data Explorer → products → Indexes tab]

Part 9: Creating the Controller and Router

Create and Provide screenshots of the Controller and Router of your newly created Model/Schema (*Add more pages if necessary*)

A controller and a router were created for each new model (category, cart, order, and review). The existing product and user controllers were updated to use the renamed models, and the new routers were registered in `index.js`. The product and user endpoints from Activity 1 still work the same way.

File	Contents
`components/controllers/categoryController.js`	getCategories, getCategory, createCategory
`components/controllers/cartController.js`	getCarts, getCart, createCart
`components/controllers/orderController.js`	getOrders, getOrder, createOrder
`components/controllers/reviewController.js`	getReviews, getReview, createReview
`components/routes/categoryRoutes.js`, `cartRoutes.js`, `orderRoutes.js`, `reviewRoutes.js`	Map each URL and HTTP method to its controller function
`components/controllers/productController.js` (updated)	Uses productModel, populates the category, checks that the category exists, and supports ?category= and ?search=
`components/controllers/userController.js` (updated)	Uses userModel
`index.js` (updated)	Registers /api/categories, /api/carts, /api/orders, and /api/reviews

API endpoints

Method	Endpoint	Controller function	Auth
GET	/api/categories	getCategories	–
GET	/api/categories/:id	getCategory	–
POST	/api/categories	createCategory	Bearer token
GET	/api/carts (optional ?user=<id>)	getCarts	–
GET	/api/carts/:id	getCart	–
POST	/api/carts	createCart	–
GET	/api/orders (optional ?user=<id>, ?status=)	getOrders	–
GET	/api/orders/:id	getOrder	–
POST	/api/orders	createOrder	–
GET	/api/reviews (optional ?product=<id>)	getReviews	–

GET	/api/reviews/:id	getReview	–
POST	/api/reviews	createReview	–
GET	/api/products (optional ?category=<id>, ?search=)	getProducts (updated)	–

How the controllers keep the data correct

- Create functions first validate the document against its schema (`document.validate()`), so missing required fields, wrong types, and values outside the allowed range return **400** with the schema's error message.
- Before saving, they check that every referenced document exists (user, product, category), so no document can point to an ObjectId that does not exist.
- createCart blocks a second cart for the same user, and createReview blocks a second review of the same product by the same user.
- createOrder copies each product's current title and price into the order and computes totalAmount on the server, so the client cannot set its own prices.
- Get functions use `populate()` to replace the stored ObjectIds with the referenced data (for example the user's name or the product's title).
- An invalid ObjectId in the URL returns **400**, and an ObjectId that does not exist returns **404**.

[Paste screenshot here: categoryController.js]

[Paste screenshot here: categoryRoutes.js]

[Paste screenshot here: cartController.js]

[Paste screenshot here: cartRoutes.js]

[Paste screenshot here: orderController.js]

[Paste screenshot here: orderRoutes.js]

[Paste screenshot here: reviewController.js]

[Paste screenshot here: reviewRoutes.js]

[Paste screenshot here: productController.js (updated)]

[Paste screenshot here: index.js (updated)]

Part 10: Enhancement Activity

Do the following:

- Using MongoDB Compass/MongoDB Atlas or Postman, manually insert at least three (3) sample documents into each collection.
- Verify that Validation rules are enforced, required fields are recognized, and relationships are stored correctly.
- Provide screenshots (*Add more pages if necessary*)

The sample documents were inserted through the API with **Postman**, so every document passed through the Mongoose validation rules (inserting directly in Compass or Atlas would skip them). The requests are saved in the Postman collection **Activity2-CTADWEBL** (`Activity2-CTADWEBL.postman_collection.json`). Every create request saves the new `_id` into a collection variable, so the later requests use real ObjectIds for their references.

Before the inserts, `npm run seed` updated the 8 starter products from the previous activity: it created their 8 categories and changed each product's category from plain text to the ObjectId of its Category document, keeping the products' original _id values.

Sample documents (3 per collection)

#	Request	Data sent	Result
1.1–1.3	POST /api/users/signup	Maria Santos, Jose Reyes, Ana Garcia (@nu.edu.ph emails, password pass123)	201 Created; the password is hashed and not returned
1.4	POST /api/users/signin	Maria's email and password	200 OK; returns the JWT token used for the protected routes
2.1–2.3	POST /api/categories (Bearer token)	Uniforms, Books, Sports (each with a description)	201 Created
3.1–3.3	POST /api/products (Bearer token)	NU Uniform Polo (Uniforms, 650), Web Programming Guide (Books, 450), Bulldogs Jersey (Sports, 899)	201 Created; category is stored as an ObjectId
4.1–4.3	POST /api/carts	Maria: 2 polos + 1 guide; Jose: 1 jersey; Ana: 1 polo + 2 jerseys	201 Created
5.1–5.3	POST /api/orders	Maria: 2 polos (GCash); Jose: 1 guide + 1 jersey (Cash on Delivery); Ana: 2 jerseys (Card)	201 Created; totals 1300, 1349 and 1798 computed by the server
6.1–6.3	POST /api/reviews	Maria → polo (5), Jose → guide (4), Ana → jersey (5)	201 Created

Relationships stored correctly

#	Request	What the response shows
2.4	GET /api/categories	The 3 new categories are listed together with the 8 starter categories.
3.4	GET /api/products?category={{categoryId3}}	The stored category ObjectId is populated as { _id, name: "Sports" }.
3.5	GET /api/products?search=jersey	Bulldogs Jersey is found through the text index.
4.4	GET /api/carts?user={{userId1}}	The cart's user (name, email) and each item's product (title, price, stock) are populated.
5.4	GET /api/orders?user={{userId1}}	The order's user is populated; the items keep the title and price copied at checkout.
6.4	GET /api/reviews?product={{productId1}}	The reviewer's name and the product title are populated.

Validation rules enforced (all return 400 Bad Request)

#	Invalid data sent	Error message returned
1.5	Sign up with email "test-user-at-nu"	User validation failed: email: Please enter a valid email address
2.5	Category without a name	Category validation failed: name: Category name is required
2.6	Second category named "Books"	E11000 duplicate key error ... index: name_1 dup key: { name: "Books" }

3.6	Product without a price	Product validation failed: price: Price is required
3.7	Product with stock "Sold out"	Product validation failed: stock: Sold out is not a valid stock status
3.8	Product with category 000000000000000000000000	Category not found
4.5	Cart item with quantity 0	Cart validation failed: items.0.quantity: Quantity must be at least 1
4.6	Second cart for Maria	This user already has a cart
5.5	Order with an empty items array	Order validation failed: items: Order must have at least one item
5.6	Order with phone "12345"	Order validation failed: shippingAddress.phone: Phone number must be 11 digits and start with 09
6.5	Review with rating 6	Review validation failed: rating: Rating cannot be more than 5
6.6	Maria reviews the NU Uniform Polo again	This user already reviewed this product

Every request in the collection also has Postman tests (shown in the **Test Results** tab), and all of them pass: the valid documents return 201 or 200, and every invalid request is rejected with the expected error message.

Users

[Paste screenshot here: 1.1–1.3 Sign up (Maria, Jose, Ana) – 201 Created]

[Paste screenshot here: 1.5 Invalid email – 400 Bad Request]

[Paste screenshot here: Atlas Data Explorer – users collection]

Categories

[Paste screenshot here: 2.1–2.3 Create categories – 201 Created]

[Paste screenshot here: 2.5 / 2.6 Missing name and duplicate name – 400 Bad Request]

[Paste screenshot here: Atlas Data Explorer – categories collection]

Products

[Paste screenshot here: 3.1–3.3 Create products – 201 Created]

[Paste screenshot here: 3.4 Products in a category with the populated category]

[Paste screenshot here: 3.6–3.8 Missing price, invalid stock, category not found – 400 Bad Request]

[Paste screenshot here: Atlas Data Explorer – products collection]

Carts

[Paste screenshot here: 4.1–4.3 Create carts – 201 Created]

[Paste screenshot here: 4.4 Maria's cart with the populated user and products]

[Paste screenshot here: 4.5 / 4.6 Quantity 0 and second cart – 400 Bad Request]

[Paste screenshot here: Atlas Data Explorer – carts collection]

Orders

[Paste screenshot here: 5.1–5.3 Create orders – 201 Created (totals computed)]

[Paste screenshot here: 5.4 Maria's orders with the populated user]

[Paste screenshot here: 5.5 / 5.6 No items and invalid phone – 400 Bad Request]

[Paste screenshot here: Atlas Data Explorer – orders collection]

Reviews

[Paste screenshot here: 6.1–6.3 Create reviews – 201 Created]

[Paste screenshot here: 6.4 Reviews of the NU Uniform Polo with the populated user and product]

[Paste screenshot here: 6.5 / 6.6 Rating 6 and duplicate review – 400 Bad Request]

[Paste screenshot here: Atlas Data Explorer – reviews collection]

Conclusion

The Bulldogs Exchange database was designed as six collections and implemented as Mongoose schemas with data types, validation rules, relationships, and indexes. Valid documents were saved with their references stored as ObjectIds and returned with populate(). Invalid documents (missing required fields, values outside the allowed range or enum, duplicate unique values, and references to documents that do not exist) were rejected with clear 400 errors. This confirms that the database keeps its data consistent and that the indexes support the queries the system runs most often.

Activity Rubrics

Criteria	Beginning (1)	Developing (2)	Proficient (3)	Excellent (4)	Pts
Business Requirement Analysis and Data Modeling	Demonstrated limited understanding of the Bulldogs Exchange requirements and identified few database entities or relationships correctly.	Identified the major collections and relationships but with several inaccuracies or incomplete justifications.	Correctly identified the required collections, attributes, and most relationships with appropriate justifications.	Thoroughly analyzed business requirements and accurately identified collections, attributes, relationships, and appropriate data modeling strategies	
Schema Development	Schemas were incomplete or contained major errors in field definitions, data types, or validation rules.	Schemas were partially implemented but contained several errors in structure, validation, or relationships.	Schemas were correctly implemented with appropriate data types, validations, and relationships, with only minor issues.	Implemented Mongoose schemas following best practices, with proper validations, data types, and collection relationships.	
Database Optimization	Indexes were missing or incorrectly implemented, with little understanding of	Some indexes were implemented but with incomplete or incorrect justifications.	Appropriate indexes were implemented for most searchable fields with satisfactory	Applied and justified indexes to improve query performance and database optimization.	

	their purpose.		explanations.		
Database Validation and Testing	Database models failed to execute correctly or sample data could not be inserted successfully.	Database models executed with several errors, and only some collections or relationships were successfully validated.	Database models executed successfully, and sample documents demonstrated correct validation and relationships with only minor issues.	Successfully executed database models with correct validations and sample documents demonstrating data integrity.	
Organization, Documentation, and Submission Requirements	Required documentation and screenshots were largely incomplete, missing, or poorly organized.	Most required documentation was submitted but contained several missing or incomplete components.	Required documentation, screenshots, and diagrams were complete with only minor omissions or formatting issues.	Submitted complete, organized, and professional documentation, including the Collection Relationship Diagram (CRD), screenshots, and project files.	
Total Score					